

100 .NET Core, OOP, Microservices & LINQ Interview Questions

Interview preparation guide • 100 questions with concise answers • .NET Developer interview revision

Section 1 — .NET Core / ASP.NET Core

1. What is .NET Core / modern .NET?

Answer: A cross-platform, open-source runtime and development platform for building web, cloud, desktop, services, and console applications. Modern .NET combines the former .NET Core and .NET Framework product lines under the .NET name.

2. What is the difference between .NET Framework and modern .NET?

Answer: .NET Framework is the older Windows-focused platform. Modern .NET is cross-platform, high-performance, open-source, and supports side-by-side runtime versions.

3. What is Kestrel?

Answer: Kestrel is the cross-platform web server used by ASP.NET Core. It can run directly or behind a reverse proxy such as IIS, Nginx, or Apache.

4. What is middleware in ASP.NET Core?

Answer: Middleware is software assembled into an HTTP request pipeline. Each component can inspect or modify the request/response and decide whether to call the next component.

5. What is dependency injection?

Answer: Dependency injection supplies an object's dependencies from outside instead of the object constructing them itself. ASP.NET Core provides a built-in DI container.

6. What are Transient, Scoped, and Singleton lifetimes?

Answer: Transient creates a new instance each time requested; Scoped normally creates one instance per request scope; Singleton creates one instance for the application's lifetime.

7. Why should DbContext usually be Scoped?

Answer: DbContext represents a unit of work and is not thread-safe. A scoped lifetime normally aligns it with one HTTP request/unit of work.

8. What is configuration in ASP.NET Core?

Answer: Configuration combines settings from sources such as JSON files, environment variables, command-line arguments, user secrets, and external providers.

9. What is appsettings.json?

Answer: A conventional JSON configuration file used for application settings. Environment-specific files such as appsettings.Production.json can override base settings.

10. What is the Options pattern?

Answer: The Options pattern binds configuration sections to strongly typed classes, commonly using IOptions, IOptionsSnapshot, or IOptionsMonitor.

11. What is routing?

Answer: Routing maps an incoming URL and HTTP method to an endpoint such as a controller action, minimal API handler, or Razor endpoint.

12. What is attribute routing?

Answer: Attribute routing defines routes directly on controllers/actions using attributes such as [Route], [HttpGet], and [HttpPost].

13. What is model binding?

Answer: Model binding converts HTTP request data such as route values, query strings, form fields, and JSON bodies into .NET objects or parameters.

14. What is model validation?

Answer: Model validation checks incoming data against validation rules, such as DataAnnotations or custom validators, before application logic processes it.

15. What is IActionResult?

Answer: IActionResult represents an HTTP action result and allows an action to return different response types such as Ok, NotFound, BadRequest, or Created.

16. What is async/await?

Answer: async/await provides a convenient way to write asynchronous code. It allows an operation to yield control while awaiting I/O instead of blocking a thread.

17. Why use async APIs for database and HTTP calls?

Answer: Database and network operations are I/O-bound. Async APIs allow threads to do other work while waiting, improving scalability under load.

18. What is cancellation in ASP.NET Core?

Answer: CancellationToken allows long-running or asynchronous operations to stop when work is no longer needed, such as when a client disconnects or a request is aborted.

19. What is logging?

Answer: Logging records diagnostic information about application behavior, failures, and performance. ASP.NET Core provides Microsoft.Extensions.Logging abstractions.

20. What is global exception handling?

Answer: It is a centralized mechanism that catches unhandled exceptions, logs them, and returns consistent safe error responses, often using exception-handling middleware.

21. What is authentication?

Answer: Authentication determines who a caller is, commonly using cookies, JWT bearer tokens, OpenID Connect, or another identity mechanism.

22. What is authorization?

Answer: Authorization determines what an authenticated caller is allowed to do, using roles, claims, policies, or resource-based rules.

23. What is CORS?

Answer: Cross-Origin Resource Sharing is a browser security mechanism that controls whether a web application from one origin may call a server on another origin.

24. What is OpenAPI/Swagger?

Answer: OpenAPI is a specification for describing HTTP APIs. Swagger tooling can generate interactive API documentation and client/server artifacts from an OpenAPI document.

25. What is a DTO?

Answer: A Data Transfer Object is a model designed for transferring data across a boundary. DTOs help avoid exposing internal domain/entity structures directly.

26. What is Entity Framework Core?

Answer: EF Core is a modern object-relational mapper for .NET that lets developers work with databases using .NET objects and LINQ.

27. What is DbContext?

Answer: DbContext represents a session with the database and coordinates querying, change tracking, saving changes, and database configuration.

28. What is migration in EF Core?

Answer: A migration is a versioned description of schema changes that can be applied to a database as the model evolves.

29. What is tracking vs AsNoTracking?

Answer: Tracking queries keep entity state in the DbContext for updates. AsNoTracking queries skip change tracking and are often useful for read-only results.

30. What is IQueryable?

Answer: IQueryable represents a query that can be translated and executed by a query provider, such as EF Core. Operators can build an expression tree before execution.

Section 2 — OOP & C# Fundamentals

31. What is OOP?

Answer: Object-oriented programming organizes software around objects containing state and behavior. Core concepts include encapsulation, abstraction, inheritance, and polymorphism.

32. What is encapsulation?

Answer: Encapsulation hides an object's internal state and implementation details behind a controlled public interface.

33. What is abstraction?

Answer: Abstraction exposes essential behavior while hiding unnecessary implementation details, commonly through interfaces or abstract classes.

34. What is inheritance?

Answer: Inheritance allows a derived class to reuse or specialize accessible members of a base class. It represents an is-a relationship.

35. What is polymorphism?

Answer: Polymorphism allows the same interface or base type to represent different concrete behaviors, commonly through overriding or interface implementations.

36. Interface vs abstract class?

Answer: An interface primarily defines a contract that multiple unrelated types can implement. An abstract class can provide shared state and implementation while serving as a base type.

37. Composition vs inheritance?

Answer: Composition builds objects from other objects and often gives more flexibility and lower coupling. Inheritance is appropriate when a genuine substitutable is-a relationship exists.

38. What is method overloading?

Answer: Overloading defines multiple methods with the same name but different parameter signatures within a type.

39. What is method overriding?

Answer: Overriding lets a derived class provide a new implementation for a virtual or abstract member of its base class.

40. What are virtual, override, and abstract?

Answer: virtual permits overriding; override replaces a virtual/abstract implementation in a derived class; abstract requires derived classes to implement a member.

41. What is SOLID?

Answer: SOLID is a set of five design principles: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion.

42. What is SRP?

Answer: Single Responsibility Principle says a class should have one primary reason to change, helping keep responsibilities focused.

43. What is LSP?

Answer: Liskov Substitution Principle says derived types should be usable wherever their base type is expected without breaking correctness.

44. What is DIP?

Answer: Dependency Inversion Principle says high-level policy should not depend directly on low-level details; both should depend on abstractions.

45. What is a sealed class?

Answer: A sealed class cannot be inherited. It can communicate a design restriction and may enable certain runtime optimizations.

46. Class vs struct?

Answer: A class is a reference type and normally allocated as an object on the managed heap. A struct is a value type and is copied by value unless passed by reference.

47. What is immutability?

Answer: An immutable object cannot have its observable state changed after creation. Immutability simplifies reasoning and can improve thread-safety.

48. What is dependency inversion in practical .NET?

Answer: Define interfaces for application needs, inject implementations through constructors, and keep high-level business logic independent from infrastructure details.

Section 3 — Microservices

49. What is a microservice?

Answer: A microservice is a small, independently deployable service organized around a business capability, with clear boundaries and usually its own data ownership.

50. Monolith vs microservices?

Answer: A monolith deploys a system as one application unit. Microservices split it into independently deployable services, increasing autonomy but also operational complexity.

51. What is service decomposition?

Answer: Service decomposition identifies business capabilities and boundaries so each service owns a cohesive responsibility and minimizes unnecessary coupling.

52. What is bounded context?

Answer: A bounded context is a boundary within which a domain model and its terminology have a specific, consistent meaning.

53. Why database per service?

Answer: Database ownership per service reduces coupling and allows independent evolution. Services communicate through APIs or events instead of directly sharing tables.

54. What is API Gateway?

Answer: An API Gateway is an entry point that can route requests, apply authentication, rate limits, aggregation, and other cross-cutting concerns.

55. What is service discovery?

Answer: Service discovery lets services locate healthy instances dynamically rather than relying on fixed instance addresses.

56. What is synchronous communication?

Answer: A caller sends a request and waits for a response, commonly using HTTP/REST or gRPC. It is simple but can create runtime coupling.

57. What is asynchronous communication?

Answer: A producer sends a message/event without requiring the consumer to respond immediately. Brokers and queues can improve resilience and decoupling.

58. What is a message broker?

Answer: A message broker receives, stores, routes, and delivers messages between producers and consumers, such as through queues or topics.

59. What is eventual consistency?

Answer: Eventual consistency means replicas or services may temporarily disagree but converge to a consistent state after successful propagation.

60. What is idempotency?

Answer: An operation is idempotent when repeating the same request produces the same intended result as performing it once. It is important for safe retries.

61. What is a circuit breaker?

Answer: A circuit breaker temporarily stops calls to a failing dependency after a threshold of failures, allowing recovery and preventing cascading overload.

62. What is retry with backoff?

Answer: Retry repeats transiently failed operations, usually with increasing delays and jitter. Retries should be bounded and used only for appropriate failures.

63. What is a bulkhead pattern?

Answer: Bulkheads isolate resources such as connection pools or worker capacity so failure in one area does not exhaust resources everywhere.

64. What is distributed tracing?

Answer: Distributed tracing correlates a request across multiple services using trace/span identifiers, helping locate latency and failures.

65. What is correlation ID?

Answer: A correlation ID is an identifier propagated across service calls and logs to associate events belonging to one logical request or workflow.

66. What is health check?

Answer: A health check exposes or runs checks indicating whether an application or dependency is healthy or ready to receive traffic.

67. What is observability?

Answer: Observability uses logs, metrics, and traces to understand system behavior and diagnose issues from externally visible signals.

68. What is Saga pattern?

Answer: Saga coordinates a distributed business transaction as a sequence of local transactions, using compensating actions when later steps fail.

69. What is CQRS?

Answer: Command Query Responsibility Segregation separates operations that change state from operations that read state. The models can be optimized independently.

70. What is event-driven architecture?

Answer: Components publish events representing state changes or business facts. Other components react to those events asynchronously.

71. What is an outbox pattern?

Answer: The outbox pattern stores an outgoing event in the same local database transaction as the business change, then publishes it asynchronously to avoid lost messages.

72. What is API versioning?

Answer: API versioning allows controlled evolution of an API while clients migrate, commonly through URL, header, or media-type versioning.

73. What is rate limiting?

Answer: Rate limiting controls how many requests a client can make in a period, protecting services from overload and abusive traffic.

74. What is containerization?

Answer: Containerization packages an application and its dependencies into an isolated, reproducible runtime image, commonly using Docker.

75. What is LINQ?

Answer: Language Integrated Query is a .NET feature for querying collections and queryable data using a consistent syntax.

Section 4 — LINQ & EF Core Querying

76. *IEnumerable vs IQueryable?*

Answer: IEnumerable is generally used for in-memory iteration and delegates operations to .NET objects. IQueryable builds expression trees that a provider can translate, such as SQL.

77. *What is deferred execution in LINQ?*

Answer: Deferred execution means a query is not executed when defined; it runs when the sequence is enumerated or materialized.

78. *What is immediate execution?*

Answer: Immediate execution occurs when an operator materializes or aggregates a query, such as ToList, ToArray, Count, Sum, or First.

79. *First vs FirstOrDefault?*

Answer: First returns the first element and throws if the sequence is empty. FirstOrDefault returns the default value when no element exists.

80. *Single vs SingleOrDefault?*

Answer: Single requires exactly one matching element and throws if zero or more than one exists. SingleOrDefault allows zero but still throws for multiple matches.

81. *Where vs Select?*

Answer: Where filters elements. Select projects each element into another shape or value.

82. *SelectMany?*

Answer: SelectMany flattens nested sequences into one sequence, useful when each parent contains a collection of children.

83. *Any vs Count > 0?*

Answer: Any expresses an existence check and can stop after finding one match. Count may need to process more elements, so Any is usually preferable for existence.

84. *OrderBy vs ThenBy?*

Answer: OrderBy starts primary sorting. ThenBy adds secondary sorting to the existing ordered sequence.

85. *GroupBy?*

Answer: GroupBy partitions elements into groups based on a key, allowing aggregation or processing per group.

86. *Join vs GroupJoin?*

Answer: Join combines matching elements from two sequences. GroupJoin associates each outer element with a collection of matching inner elements.

87. *Contains in LINQ?*

Answer: Contains checks whether a sequence contains a specified value, using equality semantics or a supplied comparer where supported.

88. *Any with predicate?*

Answer: Any(predicate) checks whether at least one element satisfies the condition and is generally efficient for existence tests.

89. *All in LINQ?*

Answer: All returns true when every element satisfies a predicate. For an empty sequence, All returns true.

90. *Distinct?*

Answer: Distinct removes duplicate elements according to equality comparison; DistinctBy can remove duplicates based on a selected key in supported modern .NET versions.

91. Skip and Take?

Answer: Skip ignores a specified number of elements; Take returns a specified number. They are commonly used for paging.

92. How do you optimize LINQ queries?

Answer: Filter early, project only needed columns, avoid unnecessary materialization, use appropriate indexes, inspect generated SQL for database queries, and avoid N+1 access patterns.

93. What is N+1 query problem?

Answer: It occurs when one query loads parent records and then an additional query runs for each parent, causing many database round trips. Projection or eager/batched loading can help.

94. What is projection in EF Core?

Answer: Projection selects only the required fields into a DTO or anonymous type, reducing transferred data and avoiding unnecessary entity materialization.

95. What is pagination?

Answer: Pagination returns data in manageable pages using mechanisms such as Skip/Take or keyset/seek pagination. For large datasets, keyset pagination can be more efficient.

96. What is AsSplitQuery in EF Core?

Answer: AsSplitQuery can execute related collection loading as multiple SQL queries instead of one large joined query, potentially reducing cartesian explosion.

97. What is a common microservices interview trade-off?

Answer: Microservices improve independent deployment and team autonomy but add network latency, distributed failure modes, deployment complexity, observability needs, and data consistency challenges.

98. What is Minimal API in ASP.NET Core?

Answer: Minimal APIs provide a lightweight way to define HTTP endpoints with less ceremony than traditional controllers, while still supporting dependency injection, routing, authentication, validation, and middleware.

99. What is the difference between REST and gRPC?

Answer: REST commonly uses HTTP semantics and JSON and is broadly interoperable. gRPC uses HTTP/2 and Protocol Buffers, offering strongly typed contracts and efficient service-to-service communication.

100. What is Clean Architecture?

Answer: Clean Architecture separates business rules from frameworks, UI, databases, and infrastructure. Dependencies point inward toward the application/domain core, improving testability and maintainability.

Note: Answers are intentionally concise for interview revision. For version-specific behavior, confirm details against the documentation for the .NET version used by the employer.